

TECNOLOGÍAS DE LA INFORMACIÓN Desarrollo de Sistemas Informáticos 4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: ALBA ALEXNDRA SILVA PILA

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: DESARROLLO DEL BACKEND Y BASE DE DATOS (API REST)

PERIODO ABRIL – AGOSTO DE 2026



Actividades a desarrollar en la Unidad 4

Actividad # 8

DESARROLLO DEL BACKEND Y BASE DE DATOS (API REST)

Instrucciones

La presente actividad tiene como finalidad desarrollar el backend de un Sistema de Gestión de Incidentes HelpDesk, mediante la implementación de una API REST utilizando Node.js, Express.js y SQLite, permitiendo gestionar la información correspondiente a los tickets registrados dentro del sistema.

Durante el desarrollo del proyecto se implementó una estructura organizada del backend, aplicando una distribución modular mediante carpetas destinadas a la configuración, controladores, modelos, rutas, middleware y datos iniciales del sistema.

Para el almacenamiento de la información se configuró una base de datos utilizando SQLite, integrándola mediante Sequelize ORM, con el objetivo de establecer una comunicación adecuada entre la aplicación y los datos almacenados.

Además, se desarrollaron los servicios principales de la API REST aplicando operaciones CRUD (Create, Read, Update y Delete), permitiendo crear, consultar, actualizar y eliminar registros relacionados con la gestión de incidentes.

Finalmente, se realizaron pruebas utilizando la herramienta Postman, verificando el funcionamiento de los endpoints implementados y comprobando la correcta comunicación entre el backend y la base de datos.

Objetivos

Desarrollar el backend del Sistema de Gestión de Incidentes mediante la implementación de una API REST utilizando Node.js, Express y SQLite.

- Construir la estructura principal del backend del sistema HelpDesk.
- Configurar la base de datos SQLite.
- Implementar modelos, controladores y rutas.
- Desarrollar operaciones CRUD para la gestión de tickets.
- Validar el funcionamiento de la API mediante Postman.
- Publicar el proyecto mediante GitHub.



Herramientas Utilizadas

Las principales tecnologías utilizadas fueron:

Node.js

Entorno de ejecución utilizado para desarrollar la aplicación del lado del servidor, permitiendo ejecutar código JavaScript para la construcción del backend.

Express.js

Framework utilizado para la creación de la API REST, facilitando la gestión de rutas, solicitudes HTTP y respuestas del servidor.

Sequelize ORM

Herramienta utilizada para establecer la comunicación entre la aplicación y la base de datos, permitiendo administrar los modelos y operaciones sobre la información almacenada.

SQLite

Sistema gestor de base de datos utilizado para almacenar la información correspondiente a los tickets del Sistema de Gestión de Incidentes.

Visual Studio Code

Editor de código utilizado para la creación, organización y administración de los archivos que conforman el proyecto.

Git

Herramienta utilizada para el control de versiones del código fuente durante el desarrollo del sistema.

GitHub

Plataforma utilizada para almacenar y gestionar el repositorio del proyecto.

Postman

Herramienta utilizada para realizar pruebas de la API REST, verificando el funcionamiento de los endpoints implementados y las operaciones CRUD.

Arquitectura del Proyecto

El proyecto fue organizado mediante una estructura modular:

ProyectoHelpDesk_Alexandra



src/

- config
- controllers
- middleware
- models
- routes
- seeders

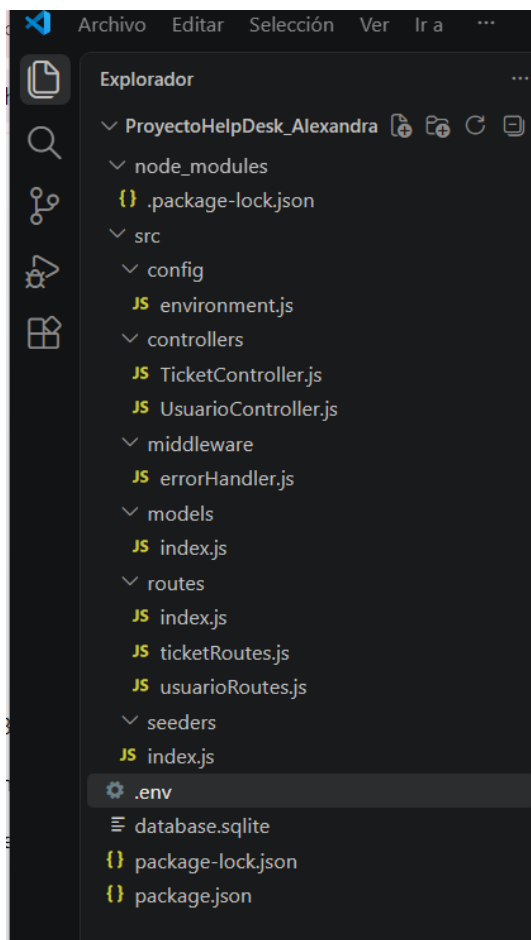
Archivos principales:

- **.env**
- package.json
- database.sqlite

Esta organización permite mantener separados los componentes del sistema y facilita la administración del código.

Evidencia:

Captura 1. Estructura del proyecto en Visual Studio Code.





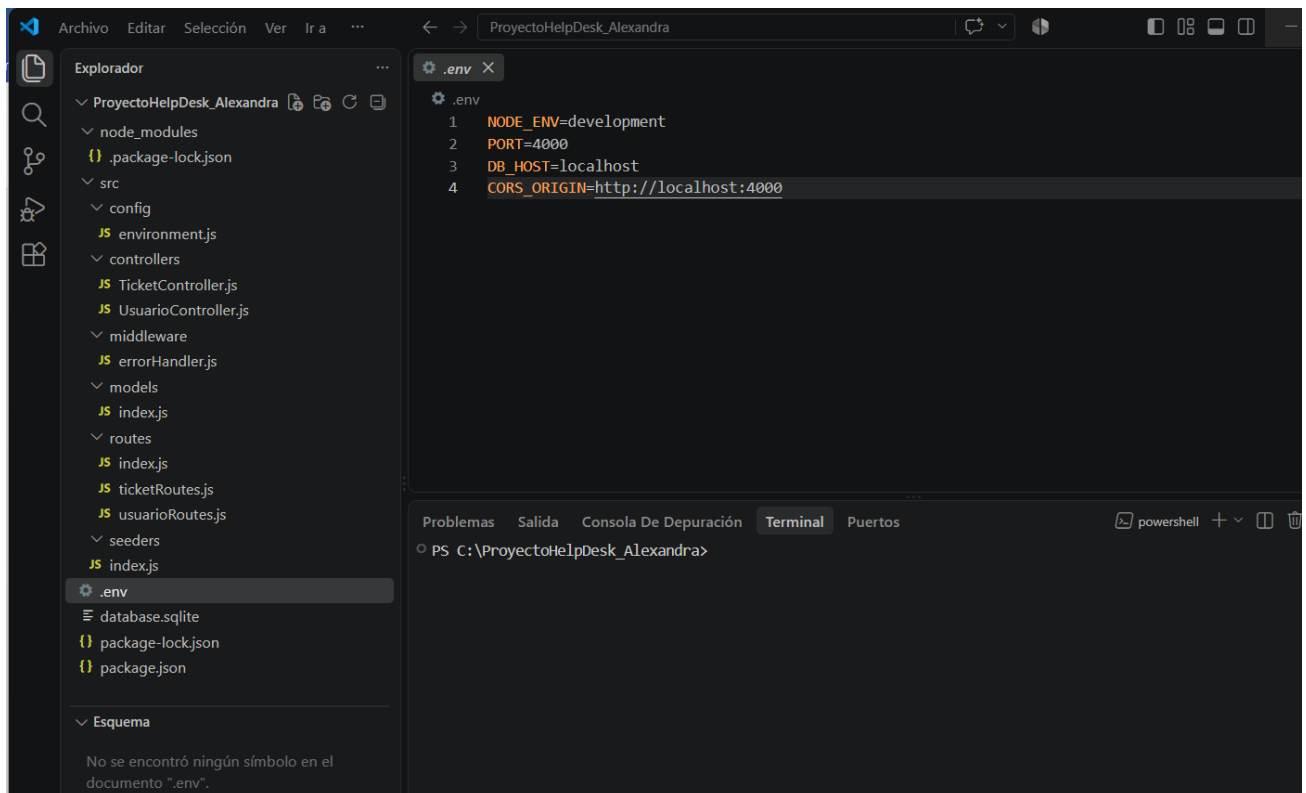
Configuración del Entorno

Para la configuración del sistema se utilizó el archivo .env con las variables necesarias para la ejecución del backend.

Configuración utilizada:

```
NODE_ENV=development  
PORT=4000  
DB_HOST=localhost  
CORS_ORIGIN=http://localhost:4000
```

Captura 2. Archivo de configuración .env.



Base de Datos

Para almacenar la información del sistema se utilizó SQLite como base de datos.

La gestión de la información se realizó mediante Sequelize ORM, permitiendo la comunicación entre los modelos del sistema y la base de datos.

Archivo utilizado:

database.sqlite



Captura 3. Base de datos y modelos del sistema.

```
src > models > JS index.js > ...
1  const { Sequelize } = require('sequelize');
2  const path = require('path');
3  require('dotenv').config();
4
5  const sequelize = new Sequelize({
6    dialect: 'sqlite',
7    storage: path.join(__dirname, '../database.sqlite'),
8    logging: false,
9  });
10
11 // MODELOS BÁSICOS
12 const Usuario = sequelize.define('Usuario', {
13   id: {
14     type: Sequelize.INTEGER,
15     primaryKey: true,
16     autoIncrement: true,
17   },
18   // ... other attributes
19 });
```

Desarrollo de la Api Rest

La API REST fue desarrollada para permitir la gestión de información del sistema mediante solicitudes HTTP.

Los métodos implementados fueron:

GET:

Permite consultar información registrada.

POST:

Permite crear nuevos tickets.

PUT:

Permite actualizar información existente.

DELETE:



Permite eliminar registros.

Captura 4. Rutas implementadas.

```
src > routes > JS index.js > ...
1  const express = require('express');
2  const router = express.Router();
3
4  const ticketRoutes = require('./ticketRoutes');
5  const usuarioRoutes = require('./usuarioRoutes');
6
7  // Health check
8  router.get('/health', (req, res) => {
9    res.json({
10      success: true,
11      message: 'Servidor operacional',
12      timestamp: new Date(),
13    });
14  });
15
16  // Rutas principales
17  router.use('/tickets', ticketRoutes);
```

Captura 5. Controladores del sistema.

```
src > controllers > JS TicketController.js > ...
4  exports.obtenerTickets = async (req, res) => {
5    try {
6      const tickets = await Ticket.findAll({
7        include: [{
8          model: Usuario,
9          as: 'usuarioReportante',
10         attributes: ['id', 'nombre', 'email'],
11       }],
12     });
13   } catch (error) {
14     res.status(500).json({
15       success: false,
16       message: 'Error al obtener tickets',
17     });
18   }
19 }
20
21 exports.reservarTicket = async (req, res) => {
22   try {
23     const { id, cantidad } = req.body;
24     const usuario = await Usuario.findById(id);
25     if (!usuario) {
26       res.status(404).json({ message: 'Usuario no encontrado' });
27     }
28     const ticket = await Ticket.findById(id);
29     if (!ticket) {
30       res.status(404).json({ message: 'Ticket no encontrado' });
31     }
32     if (ticket.cantidad < cantidad) {
33       res.status(400).json({ message: 'Cantidad no válida' });
34     }
35     ticket.cantidad -= cantidad;
36     await ticket.save();
37     res.json({ message: 'Ticket reservado exitosamente' });
38   } catch (error) {
39     res.status(500).json({ message: 'Error al reservar ticket' });
40   }
41 }
```



Operaciones Crud

Dentro de la API REST se implementaron las operaciones CRUD:

Create:

Permite registrar nuevos datos.

Read:

Permite consultar información existente.

Update:

Permite modificar registros.

Delete:

Permite eliminar información.

Pruebas Realizadas con Postman

Evidencia de pruebas de la API REST mediante Postman

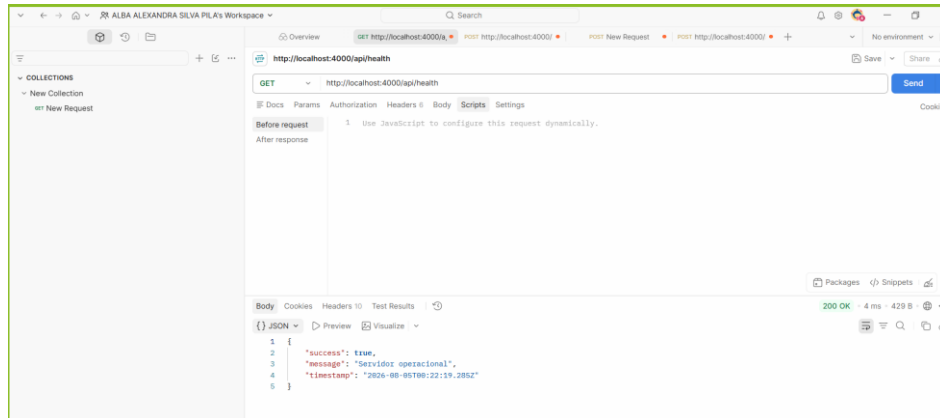
Se creó una colección de pruebas en Postman para verificar el correcto funcionamiento de los servicios REST implementados en la API Help Desk. La colección contiene las operaciones principales del sistema utilizando los métodos HTTP correspondientes:

Las pruebas de funcionamiento fueron realizadas mediante Postman, verificando los endpoints desarrollados.

Se realizaron pruebas de:

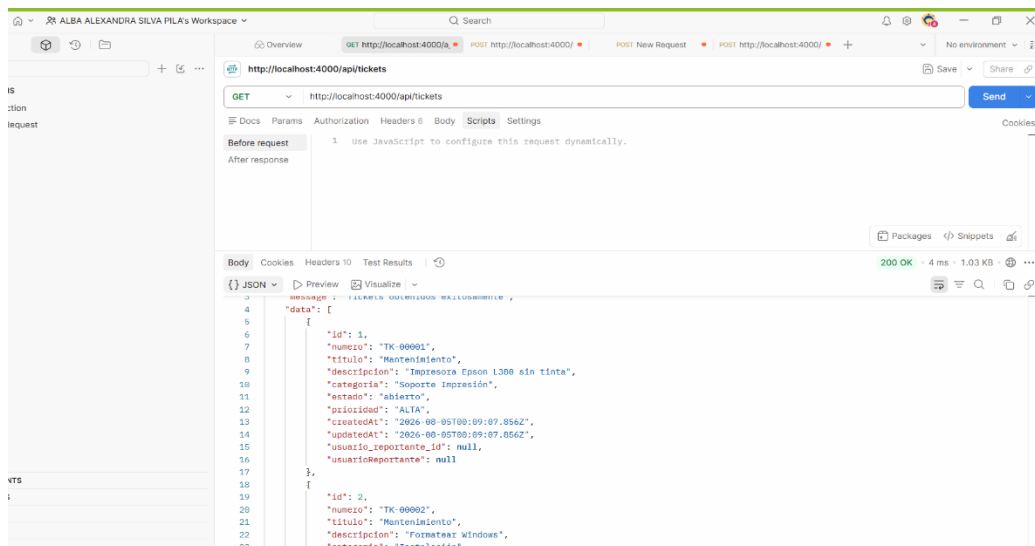
- Endpoint /api/health.
- Solicitudes GET.
- Solicitudes POST.
- Solicitudes PUT.
- Solicitudes DELETE.

Captura 6. Prueba del endpoint /api/health.

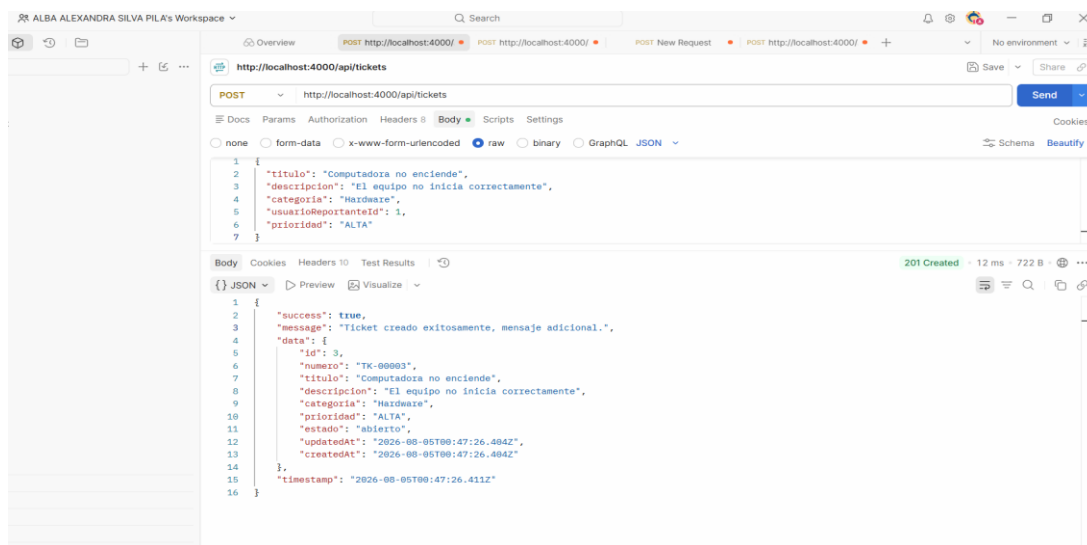


Comprobar listado de tickets

http://localhost:4000/api/tickets



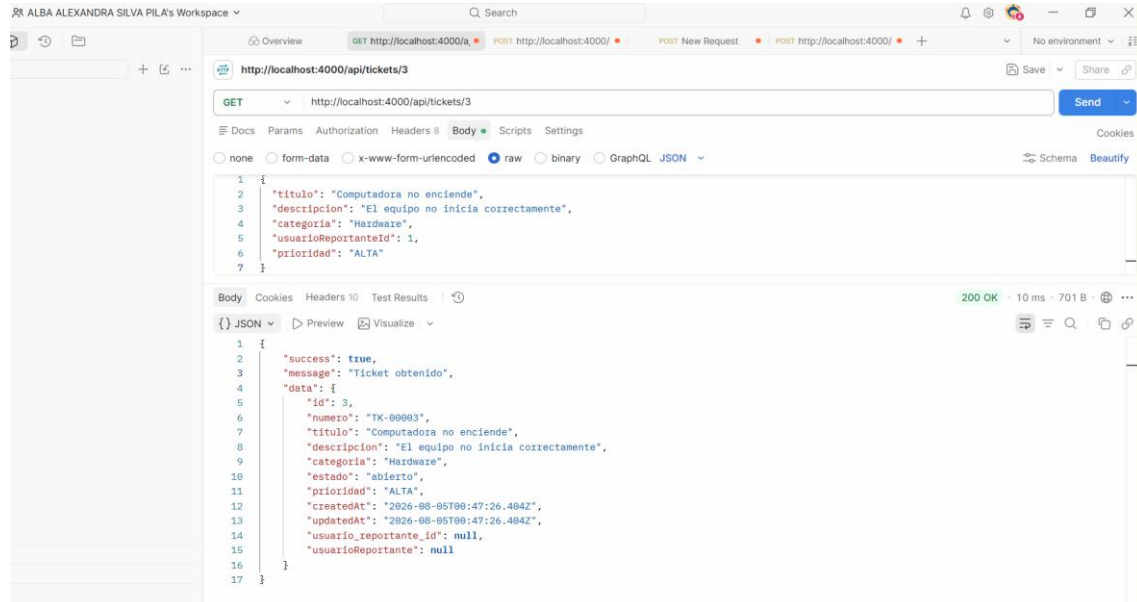
Probar POST (Crear ticket)





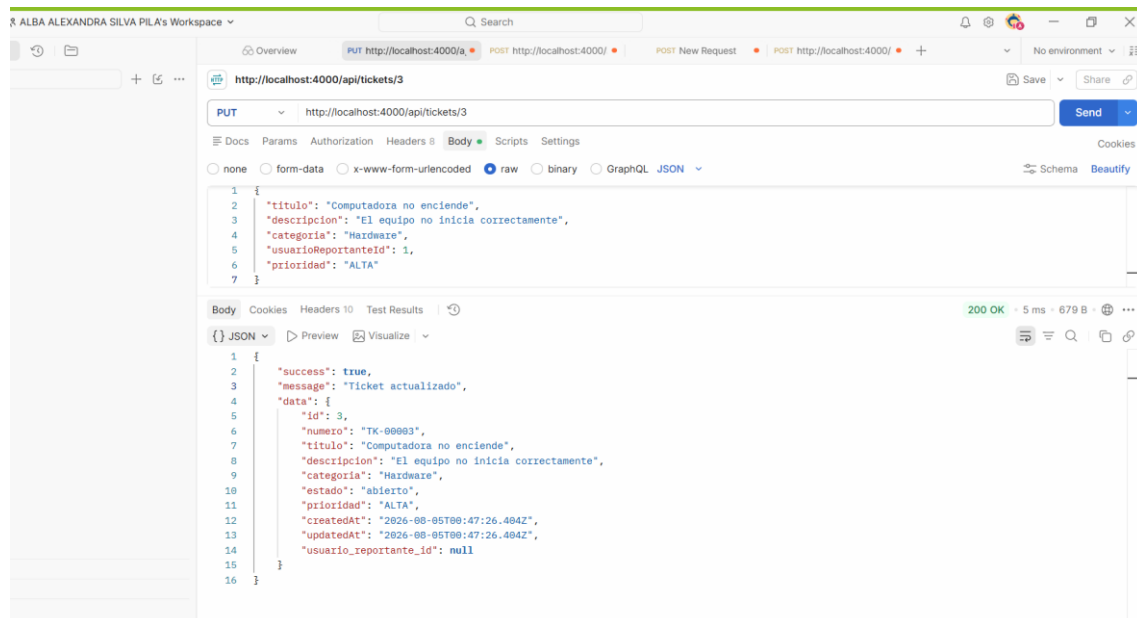
Consultar el ticket creado por ID

Consulta de ticket específico mediante GET /api/tickets/3



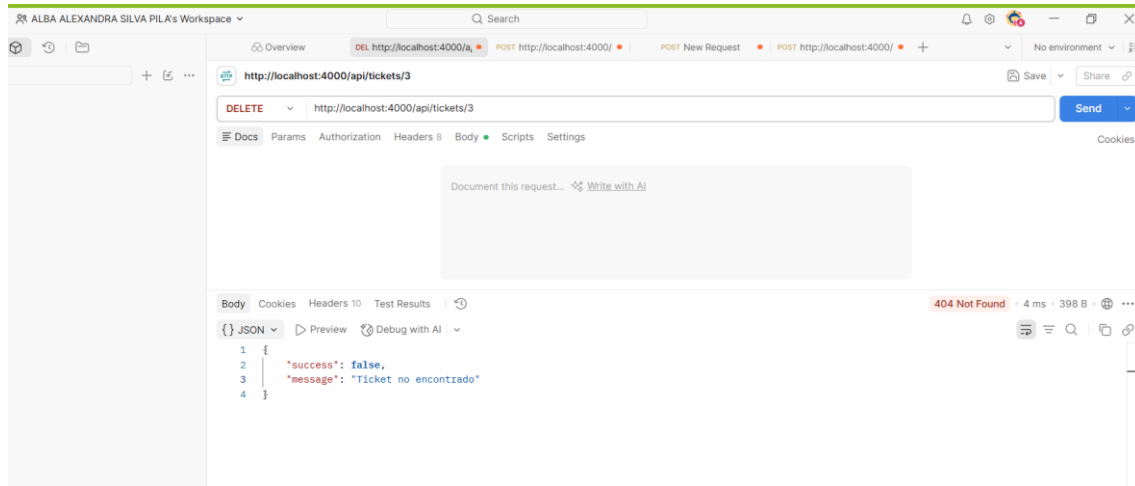
Actualizar ticket

Actualización de ticket mediante PUT /api/tickets/3



Eliminar ticket

Eliminación de ticket mediante DELETE /api/tickets/3



Resultados Obtenidos

Como resultado de la actividad se obtuvo un backend funcional con una estructura organizada, base de datos SQLite configurada y servicios REST disponibles para la gestión de incidentes.

Los principales resultados fueron:

- Implementación del backend.
- Configuración de la base de datos.
- Desarrollo de la API REST.
- Implementación de operaciones CRUD.
- Validación mediante Postman.

Conclusiones

El desarrollo de esta actividad permitió aplicar conocimientos relacionados con la creación de servicios backend, manejo de bases de datos y desarrollo de APIs REST.

La implementación realizada mediante Node.js, Express, Sequelize y SQLite permitió construir una estructura organizada para la gestión de incidentes del sistema HelpDesk.

Las pruebas realizadas mediante Postman permitieron verificar el correcto funcionamiento de los servicios implementados.

Publicación del Proyecto mediante GitHub

Para el control de versiones y almacenamiento del código fuente desarrollado se utilizó la plataforma GitHub, el proyecto del Sistema de Gestión de Incidentes HelpDesk fue publicado en un repositorio remoto, permitiendo mantener una copia del backend y facilitar la administración del código durante el desarrollo.

El repositorio contiene la estructura completa del proyecto, incluyendo:

- Configuración del servidor backend.



- Controladores para la gestión de tickets.
- Modelos de base de datos mediante Sequelize ORM.
- Rutas de la API REST.
- Middleware utilizado.
- Archivos de configuración.
- Base de datos SQLite.

Repositorio GitHub del proyecto:

<https://github.com/asilva6635-glitch/-Backend.git>

Captura Repositorio del proyecto en GitHub.

```
Símbolo del sistema
C:\ProyectoHelpDesk_Alexandra>git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/-Backend/pull/new/develop
remote:
To https://github.com/asilva6635-glitch/-Backend.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

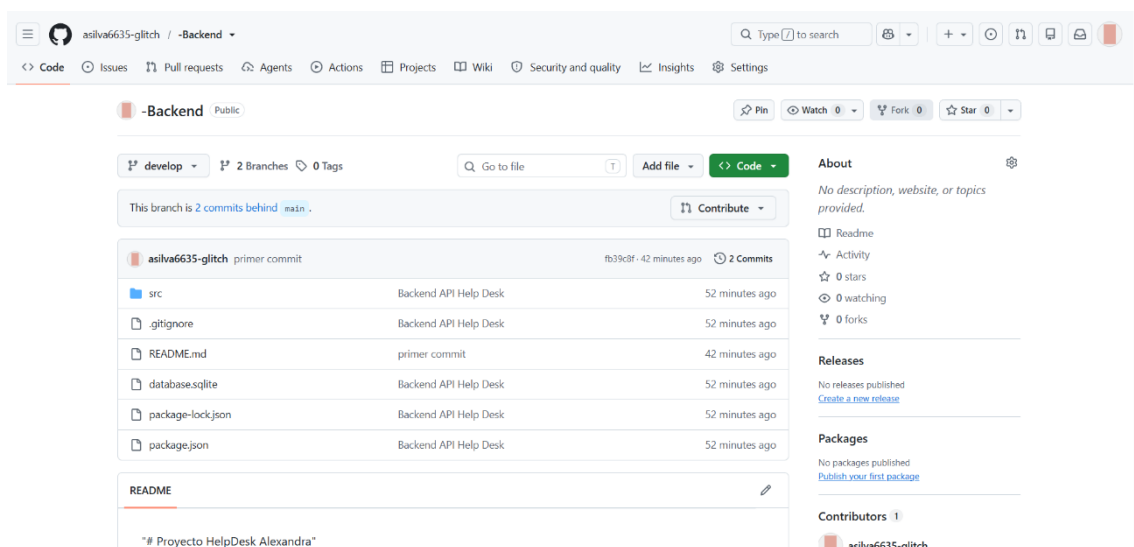
C:\ProyectoHelpDesk_Alexandra>git add .

C:\ProyectoHelpDesk_Alexandra>git commit -m "Backend API Help Desk - Feature"
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean

C:\ProyectoHelpDesk_Alexandra>git push -u origin feature/backend-api -f
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/backend-api' on GitHub by visiting:
remote:   https://github.com/asilva6635-glitch/-Backend/pull/new/feature/backend-api
remote:
To https://github.com/asilva6635-glitch/-Backend.git
 * [new branch]      feature/backend-api -> feature/backend-api
branch 'feature/backend-api' set up to track 'origin/feature/backend-api'.

C:\ProyectoHelpDesk_Alexandra>
```





github.com/asilva6635-glitch/-Backend/tree/feature/backend-api

asilva6635-glitch / -Backend

Type to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

-Backend Public

Pin Watch 0 Fork 0

feature/backend-api 3 Branches 0 Tags

Go to file

Add file

Code

About

This branch is 2 commits behind main.

Contribute

asilva6635-glitch primer commit fb39c8f · 47 minutes ago 2 Commits

src	Backend API Help Desk	1 hour ago
.gitignore	Backend API Help Desk	1 hour ago
README.md	primer commit	47 minutes ago
database.sqlite	Backend API Help Desk	1 hour ago
package-lock.json	Backend API Help Desk	1 hour ago
package.json	Backend API Help Desk	1 hour ago

README

"# Proyecto HelpDesk Alexandra"

About

No description, website, or provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

asilva6635-glitch